

CHAPTER 08

08. MVC 게시판 + 유저 — 코드를 역할별로 나눈다

Node.js · Express · NestJS 강의 · 2026

06장과 07장의 게시판은 한 파일에 모든 것이 들어 있었다. 라우팅도, 요청 검증도, DB 질의도 같은 함수 안에 뒤섞여 있다. 글이 짧을 땐 그게 더 빨라 보인다. 그런데 기능이 늘고 파일이 200줄을 넘기면 이야기가 달라진다. "게시물 수정"을 고치려는데 그 코드가 라우트·검증·DB 사이 어디쯤 있는지부터 찾아야 하고, 한 줄을 바꾸면 엉뚱한 데가 같이 깨진다.

이 장에서는 같은 게시판을 **역할별 계층**으로 나눈다. URL을 연결하는 자리(라우트), 요청과 응답을 다루는 자리(컨트롤러), 실제 로직을 처리하는 자리(서비스), DB를 정의하는 자리(모델)를 각각 다른 곳에 둔다. 이렇게 나누는 방식을 **MVC**라 부른다. 그리고 작성자를 단순 문자열이 아니라 별도의 유저 테이블로 떼어 내, 게시물과 유저를 관계로 잇는다.

화면(HTML)은 여전히 다루지 않는다. 이 과정의 서버는 처음부터 끝까지 JSON API다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- MVC 4계층 — 라우트 → 컨트롤러 → 서비스 → 모델이 각각 무엇을 맡는가
- 요청 하나가 네 계층을 어떻게 차례로 지나는지(게시물 목록·생성으로 추적)
- `express.Router()` 로 자원별 라우터를 만들고 `app.use` 로 마운트하는 법
- 모델 사이의 관계 설정 (`hasMany` · `belongsTo`)과, 조회 시 `include` 로 작성자를 JOIN해 붙이는 법
- 각 계층의 `module.exports` / `require` 가 계층을 어떻게 잇는가

코드는 역할별로 나뉘어 있다. 한 줄로 흐름을 그리면 이렇다.

요청 → 라우트 (URL 연결) → 컨트롤러 (HTTP 처리) → 서비스 (로직) → 모델 (DB) → 응답

계층 맵은 다음과 같다.

계층	역할
진입점	미들웨어 등록, 라우터 마운트, DB 준비, 서버 시작
라우터	URL + 메서드를 컨트롤러 함수에 연결
컨트롤러	<code>req</code> 에서 값 꺼내고, 검증하고, 상태코드 정해 <code>res</code> 로 응답
서비스	비즈니스 로직 — DB 질의는 모델에 위임
모델	테이블 정의 + 관계(유저 —< 게시물) + DB 연결

핵심 개념 — 왜 계층을 나누는가

한 파일에 다 넣는 방식의 문제는 "변경"할 때 나타난다. 게시물 목록 라우트 하나에 URL 매칭, 본문 검증, DB 질의, 응답 형식이 전부 들어 있으면, 그중 무엇을 고치든 같은 덩어리를 건드리게 된다. 검증 규칙만 바꾸려다 DB 질의를 잘못 건드리는 식이다.

MVC는 그 덩어리를 관심사별로 쪼갬다. 각 계층은 자기 일만 하고, 옆 계층의 사정은 모른다.

- **라우터** — "어떤 URL의 어떤 메서드를 어떤 함수에 연결할지"만 안다. 그 함수가 안에서 무슨 일을 하는지는 모른다.
- **컨트롤러** — HTTP의 세계다. `req` 에서 값을 꺼내고, 필수값을 검증하고, 결과를 상태코드와 함께 `res` 로 내보낸다. **DB가 SQLite인지 Postgres인지는 모른다.**
- **서비스** — 비즈니스 로직의 세계다. "게시물을 작성자와 함께 조회한다", "없는 글이면 null을 돌려준다" 같은 규칙을 정한다. `req · res` 가 무엇인지 모른다.
- **모델** — DB의 세계다. 테이블 구조와 관계를 정의한다. HTTP도 로직도 모른다.

이렇게 나누면 변경이 한 계층 안에 갇힌다. 응답 형식을 바꾸면 컨트롤러만, 조회 규칙을 바꾸면 서비스만, 컬럼을 추가하면 모델만 손대면 된다. 계층 사이는 함수 호출로만 이어지므로, 한쪽을 통째로 갈아 끼워도(예: SQLite → Postgres) 나머지는 그대로다.

요청 하나가 네 계층을 지나는 길

`GET /posts` 요청 하나를 끝까지 따라가 보자. 이 그림이 이 장의 전부다.



요청은 위에서 아래로 내려가며 각 계층을 한 칸씩 지나고, 결과는 다시 거슬러 올라와 응답이 된다. 어느 계층도 두 칸 건너 뛰지 않는다. 컨트롤러가 직접 DB를 건드리거나, 서비스가 `res` 를 만지는 일은 없다. 이 한 방향 흐름이 MVC의 핵심 규칙이다.

콜아웃 — 계층을 잇는 것은 결국 `module.exports` 와 `require` 한 쌍이다. 서비스가 `module.exports = { list, ... }` 로 함수를 내보내면, 컨트롤러가 그걸 `require` 로 받아 `postService.list()` 처럼 부른다. 폴더만 나뉘었을 뿐, 함수가 함수를 부르는 평범한 자바스크립트다.

준비

설치되는 건 세 가지다. `express` (웹 프레임워크), `sequelize` (ORM), `sqlite3` (SQLite 드라이버). 예제 폴더에서 `npm install` 로 의존성을 설치한 뒤 앱을 실행하면 된다(포트 3000).

앱을 처음 띄우면 SQLite 데이터베이스 파일이 만들어지고 샘플 데이터가 한 번 들어간다. 서버를 꺼도 이 파일은 남으므로 데이터가 보존된다. 07장과 같은 Sequelize를 쓰지만, 이번 장의 주제는 ORM 문법이 아니라 구조다.

진입점 — 계층을 조립하는 자리

먼저 모든 것을 조립하는 자리부터 본다. 진입점은 계층을 직접 구현하지 않고, 만들어진 계층들을 불러다 연결만 한다.

```
// 진입점 - 요청은 [라우터 → 컨트롤러 → 서비스 → 모델] 순서로 흐른다.
const express = require("express");
const { sequelize, User, Post } = require("./models");
const userRoutes = require("./routes/userRoutes");
const postRoutes = require("./routes/postRoutes");

const app = express();
app.use(express.json());

// 자원별 라우터를 마운트한다.
app.use("/users", userRoutes);
app.use("/posts", postRoutes);
```

윗부분은 04장에서 본 뼈대 그대로다. `express()` 로 `app` 을 만들고, `app.use(express.json())` 으로 JSON 본문 파싱 미들웨어를 등록한다. 달라진 건 그 아래 두 줄이다.

- 모델 계층의 입구를 `require` 해 DB 연결과 두 모델을 한 번에 받는다. 여기서는 `sequelize` (연결)와 `User` · `Post` 만 쓴다.
- `app.use("/users", userRoutes)` — **`/users` 로 시작하는 모든 요청을 유저 라우터에게 넘긴다.** 이걸 "라우터를 마운트한다"고 한다. `/posts` 도 마찬가지로 게시물 라우터에 연결한다.

04장에서는 `app.get("/posts", ...)` 처럼 라우트를 `app` 에 직접 달았다. 여기서는 자원별로 묶은 **라우터 뭉치**를 통째로 한 경로 아래에 붙인다. 그래서 게시물 관련 라우트는 전부 게시물 라우터 한곳에 모이고, 진입점은 "어떤 경로에 어떤 라우터를 둘지"만 안다.

이어지는 부분은 서버를 띄우는 코드다.

```
// 테이블 준비(sync) 후 서버 시작. 처음이면 샘플 데이터도 한 번 넣는다. (07과 같은 async 함수 패턴)
async function main() {
  await sequelize.sync();
  if ((await User.count()) === 0) {
    const user = await User.create({ name: "김철수", email: "kim@example.com" });
    await Post.bulkCreate([
      { title: "첫 번째 글", content: "MVC 구조로 만든 게시판입니다.", userId: user.id },
      { title: "두 번째 글", content: "계층을 나눠 관리합니다.", userId: user.id },
    ]);
  }
  app.listen(3000, () => console.log("http://localhost:3000 에서 실행 중"));
}
main();
```

07장과 같은 패턴이다. `sequelize.sync()` 는 모델 정의대로 테이블이 없으면 만든다. DB 작업은 시간이 걸리므로 `await` 가 필요하고, `await` 는 `async` 함수 안에서만 쓸 수 있어 전체를 `main()` 으로 감쌌다.

- `await sequelize.sync()` — `User`, `Post` 테이블을 준비한다.
- `if ((await User.count()) === 0)` — 유저가 한 명도 없을 때(= 첫 실행)만 안쪽이 실행된다. 그래서 샘플 데이터는 딱 한 번만 들어간다.
- 샘플 게시물에 `userId: user.id` 를 넣는 데 주목한다. 작성자를 문자열 "김철수" 로 적는 게 아니라, 방금 만든 유저의 **id**로 연결한다. 이 장에서 새로 더하는 부분이 바로 이것이다.
- `app.listen(3000, ...)` — 테이블과 샘플 데이터가 준비된 **뒤에야** 서버를 연다. 순서가 중요하다.

모델 계층 — DB의 세계

요청 흐름의 끝, DB의 세계다. 컨트롤러·서비스보다 먼저 보는 이유는, 위 계층들이 모두 이 모델을 가져다 쓰기 때문이다. 모델 계층은 세 부분으로 나뉜다. 연결과 관계를 만드는 입구, 그리고 유저 테이블과 게시물 테이블을 하나씩 정의하는 자리다.

연결을 만들고 관계를 잇는 입구

```
// 모델 계층의 입구 - DB 연결을 만들고, 각 모델을 등록하고, 관계를 설정한다.
const { Sequelize } = require("sequelize");
const path = require("path");

const sequelize = new Sequelize({
  dialect: "sqlite",
  storage: path.join(__dirname, "../board.db"),
  logging: false,
});
```

`new Sequelize({})` 로 DB 연결을 만든다. `dialect: "sqlite"` 는 SQLite를 쓴다는 뜻이고, `storage` 는 그 파일 경로다. `logging: false` 는 Sequelize가 실행하는 SQL을 콘솔에 찍지 않게 한다.

07장과 달라진 첫 번째 점이 여기 있다. **연결(`sequelize`)을 이 한 곳에서만 만든다.** 그리고 각 모델에 그 연결을 넘겨준다.

```
// 연결(sequelize)은 여기 한 곳에서만 만들고, 각 모델에 넘겨 정의한다.
const User = require("./user")(sequelize);
const Post = require("./post")(sequelize);
```

`require("./user")` 는 함수를 돌려준다. 그 함수에 `(sequelize)` 를 넘겨 호출하면 비로소 `User` 모델이 완성된다. 즉 모델은 "연결을 받으면 모델을 만들어 돌려주는 공장" 형태다. 07장에서는 모델이 최상위에서 바로 `define` 을 호출했는데, 여기서는 연결을 바깥에서 주입받는다. 이렇게 하면 연결이 한 군데에서만 만들어지고, 모든 모델이 **같은 연결**을 공유한다.

그다음이 이 장의 핵심, **관계 설정**이다.

```
// 관계 - 한 User가 여러 Post를 가진다(작성자).
//   foreignKey: posts 테이블에 둘 FK 컬럼명을 직접 정한다(안 정하면 기본값은 'UserId').
//   as: 조회(include) 시 붙는 이름 - post.author, user.posts 로 접근한다.
User.hasMany(Post, { foreignKey: "userId", as: "posts" });
Post.belongsTo(User, { foreignKey: "userId", as: "author" });

module.exports = { sequelize, User, Post };
```

게시물의 작성자를 더 이상 문자열로 적지 않는다. `User` 와 `Post` 를 **관계로 잇는다.** 두 줄을 한 문장씩 읽으면 이렇다.

- `User.hasMany(Post, ...)` — "한 유저는 여러 게시물을 가진다." 1:N 관계의 1쪽 선언이다.
- `Post.belongsTo(User, ...)` — "한 게시물은 한 유저에게 속한다." 같은 관계의 N쪽 선언이다. 한 관계를 양쪽에서 한 번씩 적어 준다.

옵션 두 개가 이 관계의 동작을 정한다.

- `foreignKey: "userId"` — `posts` 테이블에 둘 외래 키(FK) 컬럼 이름이다. 게시물 행마다 "이 글의 작성자는 몇 번 유저"라는 숫자가 이 컬럼에 들어간다. 안 정하면 Sequelize가 기본값 `UserId` 로 만드는데, 여기서 `userId` 로 직접 못 박았다. 진입점의 샘플 데이터가 `userId: user.id` 를 넣은 게 바로 이 컬럼이다.
- `as: "author"` — 조회할 때 붙는 별칭이다. `belongsTo` 에 `as: "author"` 를 줬으니, 게시물을 작성자와 함께 읽으면 그 작성자가 `post.author` 로 따라온다. 별칭을 안 줬다면 기본값 `post.User` 가 됐을 것이다. "작성자"라는 뜻이 코드에 드러나도록 `author` 라는 이름을 붙였다.

마지막 `module.exports = { sequelize, User, Post }` 로 연결과 두 모델을 묶어 내보낸다. 이게 모델 계층의 공개 창구다. 진입점도, 서비스들도 모두 모델 계층의 입구를 `require` 해 여기서 필요한 것을 꺼내 쓴다.

유저 테이블

```
// User 모델 - sequelize 인스턴스를 받아 정의해서 돌려준다(모델 입구가 연결을 넘겨줌).
const { DataTypes } = require("sequelize");

module.exports = (sequelize) =>
  sequelize.define("User", {
    name: { type: DataTypes.STRING, allowNull: false },
    email: { type: DataTypes.STRING, allowNull: false, unique: true },
  });
```

`module.exports` 가 곧바로 함수다. 모델 입구가 이 함수를 호출하면서 연결을 넘기면, 그 안에서 `sequelize.define("User", {. })` 로 모델을 만들어 돌려준다. 앞에서 말한 "공장" 형태가 이 모양이다.

`User` 테이블에는 컬럼이 둘이다.

- `name` — 문자열, 빈 값 불가(`allowNull: false`).
- `email` — 문자열, 빈 값 불가에 더해 `unique: true` . 같은 이메일로는 두 명을 만들 수 없다.

`id` , `createdAt` , `updatedAt` 은 Sequelize가 자동으로 붙이므로 적지 않는다.

게시물 테이블

```
// Post 모델 - 작성자는 이제 문자열이 아니라 별도 User 테이블이다.
// Post엔 userId(FK)만 두고(연결은 모델 입구), 조회 시 include로 post.author(객체)가 붙는다.
const { DataTypes } = require("sequelize");

module.exports = (sequelize) =>
  sequelize.define("Post", {
    title: { type: DataTypes.STRING, allowNull: false },
    content: { type: DataTypes.TEXT, allowNull: false },
  });
```

유저 모델과 똑같이 연결을 받아 정의를 돌려주는 함수다. `Post` 에는 `title` (짧은 문자열 `STRING`)과 `content` (긴 글 `TEXT`)만 있다.

여기서 눈여겨볼 건 **없는 것**이다. `author` 도, `userId` 도 컬럼 목록에 적지 않았다. `userId` 는 모델 입구의 관계 설정 (`foreignKey: "userId"`)이 `posts` 테이블에 자동으로 만들어 주기 때문이다. 작성자 정보를 이 모델에 직접 적지 않고, 관계로 잇고 조회할 때 `include` 로 붙인다 — 그 `include` 가 어떻게 생겼는지는 바로 다음 서비스 계층에서 본다.

서비스 계층 — 비즈니스 로직

비즈니스 로직의 세계다. "무엇을 할지"를 정하고, 실제 DB 질의는 모델에 시킨다. `req · res` 는 이 계층에 없다. HTTP를 모르기 때문에, 이 함수들은 웹 요청뿐 아니라 배치 작업이나 테스트에서도 그대로 부를 수 있다.

게시물 서비스

```
// 서비스 계층 — 비즈니스 로직. 컨트롤러(HTTP)와 모델(DB) 사이에서 "무엇을 할지"를 담당한다.
const { Post, User } = require("../models");

// 작성자(User)를 함께(JOIN) 붙여서 돌려준다.
// as:"author" → 결과를 post.author로 꺼낸다(07의 기본값은 post.User였다).
// attributes → 작성자에서 id, name만 골라 붙인다(email 등 나머지는 제외).
const withAuthor = { include: [{ model: User, as: "author", attributes: ["id", "name"]} ] };
};
```

맨 위에서 모델 계층의 입구를 `require` 해 `Post` 와 `User` 를 받는다. 모델 계층의 공개 창구에서 꺼내 온다.

그 아래 `withAuthor` 가 이 장의 또 다른 핵심이다. 게시물을 읽을 때 **작성자 정보를 함께 JOIN해 붙이는** 조회 옵션이다. 한 조각씩 읽으면 이렇다.

- `include: [{ model: User,... }]` — "이 조회에 `User` 를 함께 가져와라." SQL로 치면 JOIN이다. 이게 없으면 게시물의 `userId` 숫자만 나오지, 작성자 이름은 안 나온다.
- `as: "author"` — 모델 입구에서 관계에 붙인 별칭과 **같은 이름**이어야 한다. 그래야 결과가 `post.author` 로 따라 온다. 별칭이 어긋나면 Sequelize가 관계를 못 찾아 에러를 낸다.
- `attributes: ["id", "name"]` — 작성자에서 `id` 와 `name` 만 골라 붙인다. `email` 같은 나머지는 응답에서 뺀다. 작성자의 이메일까지 게시물 목록에 노출할 이유가 없으니 필요한 것만 추린다.

이 한 줄을 변수로 빼 둔 덕에, 목록 조회와 단건 조회가 둘 다 같은 옵션을 재사용한다.

```
const list = () => Post.findAll({...withAuthor, order: [["id", "DESC"]]});

const get = (id) => Post.findById(id, withAuthor);

const create = ({ title, content, userId }) => Post.create({ title, content, userId });
```

- `list` — `Post.findAll` 로 전체를 읽되, `...withAuthor` 로 작성자를 붙이고 `order: [["id", "DESC"]]` 로 최신 글(id 큰 순)을 위로 올린다.

- `get` — `Post.findByPk(id, withAuthor)` 로 id 하나를 찾고, 역시 작성자를 함께 붙인다. 없으면 `null` 이 나온다.
- `create` — `Post.create` 로 새 글을 만든다. 받은 `userId` 가 그대로 FK 컬럼에 들어가, 이 글이 어떤 유저의 것인지 기록된다.

이 세 함수는 모두 `req` 를 모른다. `list` 는 인자가 없고, `get` 은 그냥 `id` 하나를, `create` 는 객체 하나를 받을 뿐이다. HTTP에서 그 값을 어떻게 꺼냈는지는 컨트롤러의 몫이고, 서비스는 깔끔한 값만 받는다.

수정과 삭제는 "먼저 찾고, 없으면 null, 있으면 처리"라는 같은 골격을 공유한다.

```
const update = async (id, fields) => {
  const post = await Post.findByPk(id);
  if (!post) return null; // 없으면 컨트롤러가 404 처리
  if (fields.title !== undefined) post.title = fields.title;
  if (fields.content !== undefined) post.content = fields.content;
  await post.save();
  return post;
};

const remove = async (id) => {
  const post = await Post.findByPk(id);
  if (!post) return null;
  await post.destroy();
  return post;
};

module.exports = { list, get, create, update, remove };
```

- `update` — id로 글을 찾고, 없으면 `null` 을 돌려준다. 있으면 보낸 필드만 바꾼다. `undefined` 로 검사하는 이유는 04장에서 짚은 것과 같다 — "값을 보냈는가"와 "값이 참인가"는 다르므로, 빈 문자열로 제목을 지우려는 의도도 살린다. `post.save()` 로 변경을 DB에 반영한다.
- `remove` — 같은 방식으로 찾고, 없으면 `null` , 있으면 `post.destroy()` 로 지운 뒤 지워진 객체를 돌려준다.

두 함수 모두 못 찾으면 에러를 던지지 않고 `null` 을 돌려준다. "찾지 못함"이 곧 "404"라고 정하는 건 HTTP의 판단이고, 그건 컨트롤러가 한다. 서비스는 결과만 정직하게 전한다. 이 역할 분담이 다음 컨트롤러에서 그대로 맞물린다.

마지막 `module.exports = { list, get, create, update, remove }` 로 다섯 함수를 내보낸다. 컨트롤러가 이 이름들을 가져다 쓴다.

유저 서비스

유저 쪽은 관계도 검증도 단순해서 훨씬 짧다. 목록·단건·생성, 셋뿐이다.

```
// 유저 서비스 계층 — 유저 관련 비즈니스 로직.
const { User } = require("../models");

const list = () => User.findAll({ order: [["id", "ASC"]] });

const get = (id) => User.findByPk(id);

const create = ({ name, email }) => User.create({ name, email });

module.exports = { list, get, create };
```

- `list` — 전체 유저를 id 오름차순(`ASC` , 먼저 가입한 순)으로 읽는다. 게시물 목록이 내림차순이던 것과 대비된다.
- `get` — id로 한 명을 찾는다. 없으면 `null` .
- `create` — `name` 과 `email` 로 새 유저를 만든다.

게시물과 달리 `include` 가 없다. 유저를 조회할 때 그 사람이 쓴 글까지 같이 끌어올 필요는 없으니, 관계를 붙이지 않은 단순 조회다.

컨트롤러 계층 — HTTP의 세계

HTTP의 세계다. `req` 에서 값을 꺼내고, 필수값을 검증하고, 서비스를 부르고, 결과를 상태코드와 함께 `res` 로 내보낸다. 실제 로직은 한 줄도 없다 — 전부 서비스에 맡긴다.

게시물 컨트롤러

```
// 컨트롤러 계층 — HTTP 요청/응답만 담당한다. 실제 일은 서비스에 맡긴다.
// 함수를 정의한 뒤 맨 아래 module.exports에 모아 두면, 라우터가 require해서 URL에 연결한다.
const postService = require("../services/postService");

const list = async (req, res) => {
  res.json(await postService.list());
};

const get = async (req, res) => {
  const post = await postService.get(req.params.id);
  if (!post) return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  res.json(post);
};
```

맨 위에서 게시물 서비스를 `require` 해 받는다. 컨트롤러는 모델을 직접 부르지 않는다. 한 칸 아래인 서비스만 안다.

- `list` — 가장 단순하다. `postService.list()` 를 `await` 로 기다려 그 결과를 `res.json(. )` 으로 그대로 내보낸다. 검증할 입력도, 따질 조건도 없다.

- `get` — `req.params.id` 로 경로 파라미터를 꺼내 서비스에 넘긴다. 서비스가 `null` 을 돌려주면(= 없는 글) **404** 로, 찾았으면 그 객체를 응답한다. 여기서 04장의 "찾지 못함 → 404" 규칙이 다시 등장한다. 단, 04장에서는 `find` 가 직접 했지만 이제는 서비스가 `null` 로 알리고 컨트롤러가 그걸 404로 번역한다.

`req.params.id` 를 `Number( )` 로 변환하지 않는 점에 주목한다. 04장에선 배열의 숫자 `id`와 비교하느라 변환이 필요했지만, 여기선 그 문자열을 Sequelize의 `findByPk` 에 그대로 넘긴다. Sequelize가 DB에 질의하며 알아서 처리하므로 변환이 필요 없다.

생성은 검증이 들어간다.

```
const create = async (req, res) => {
  const { title, content, userId } = req.body;
  if (!title || !content) {
    return res.status(400).json({ message: "title과 content는 필수입니다." });
  }
  const post = await postService.create({ title, content, userId });
  res.status(201).json(post);
};
```

- `req.body` 에서 `title`, `content`, `userId` 를 구조분해로 꺼낸다.
- `title` 이나 `content` 가 비었으면 **400 Bad Request**로 막는다. 검증은 HTTP 입력에 대한 판단이므로 컨트롤러의 일이다. 서비스까지 잘못된 값이 내려가지 않게 입구에서 거른다.
- 통과하면 `postService.create( )` 에 깔끔한 객체를 넘기고, 만들어진 글을 **201 Created**로 돌려준다.

수정과 삭제는 서비스의 `null` 신호를 404로 옮기는 같은 패턴이다.

```
const update = async (req, res) => {
  const post = await postService.update(req.params.id, req.body);
  if (!post) return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  res.json(post);
};

const remove = async (req, res) => {
  const post = await postService.remove(req.params.id);
  if (!post) return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  res.json({ message: "삭제됨", post });
};

// 이 모듈이 외부에 공개하는 함수들.
module.exports = { list, get, create, update, remove };
```

- `update` — `id`와 본문(`req.body`)을 그대로 서비스에 넘긴다. 어떤 필드만 바꿀지 고르는 건 앞서 본 서비스의 `update` 메서드가 `undefined` 검사가 한다. 서비스가 `null` 이면 404.
- `remove` — `id`를 넘겨 지우고, `null` 이면 404, 아니면 지워진 글을 응답에 담는다.

맨 아래 `module.exports = { list, get, create, update, remove }` 가 컨트롤러의 공개 창구다. 라우터가 이 다섯 이름을 가져다 URL에 연결한다.

유저 컨트롤러

```
// 유저 컨트롤러 - HTTP 요청/응답만 담당.
const userService = require("../services/userService");

const list = async (req, res) => {
  res.json(await userService.list());
};

const get = async (req, res) => {
  const user = await userService.get(req.params.id);
  if (!user) return res.status(404).json({ message: "사용자를 찾을 수 없습니다." });
  res.json(user);
};

const create = async (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ message: "name과 email은 필수입니다." });
  }
  const user = await userService.create({ name, email });
  res.status(201).json(user);
};

// 이 모듈이 외부에 공개하는 함수들 - 라우터가 이 이름으로 가져다 쓴다.
module.exports = { list, get, create };
```

게시물 컨트롤러와 구조가 똑같다. 다른 점은 셋뿐이다 — 유저는 `list` · `get` · `create` 만 있고 수정·삭제는 없다. 검증 대상이 `name` · `email` 이고, 못 찾았을 때 메시지가 "사용자를 찾을 수 없습니다."다. 컨트롤러가 얼마나 얇은지, 자원이 달라져도 같은 골격이 반복되는지 한눈에 보인다.

라우터 계층 — URL을 함수에 잇는 지도

가장 얇은 계층이다. URL + 메서드를 컨트롤러 함수에 연결하는 표 하나다.

게시물 라우터

```
// 라우터 계층 - URL과 컨트롤러를 연결한다. 진입점에서 "/posts"에 마운트된다.
const express = require("express");
const router = express.Router();
const postController = require("../controllers/postController");

router.get("/", postController.list);
router.post("/", postController.create);
router.get("/:id", postController.get);
router.put("/:id", postController.update);
router.delete("/:id", postController.remove);

module.exports = router;
```

- `express.Router()` — 라우트만 따로 모으는 작은 라우터를 만든다. `app` 처럼 `.get` · `.post` 를 달 수 있지만, 자체로 서버를 띄우진 않는다. 다 만든 뒤 진입점이 `app.use("/posts", ...)` 로 마운트한다.
- 게시물 컨트롤러를 `require` 해 공개 함수들을 받는다.
- `router.get("/", postController.list)` — 여기 경로가 `"/` 인 데 주의한다. 진입점에서 이 라우터를 `/posts` 에 마운트했으므로, 여기의 `"/` 는 실제로 `/posts` 다. `("/:id)` 는 `/posts/:id` 가 된다. 라우터 안의 경로는 마운트 지점 뒤에 이어 붙는다.

다섯 줄이 04장에서 본 다섯 CRUD 라우트와 정확히 같다. 다만 핸들러를 여기 직접 적지 않고 이미 만들어 둔 컨트롤러 함수를 가리키기만 한다. 라우터는 "어떤 URL이 어떤 함수로 가는지"의 지도일 뿐, 그 함수가 무슨 일을 하는지는 모른다.

유저 라우터

```
// 유저 라우터 - 진입점에서 "/users"에 마운트된다.
const express = require("express");
const router = express.Router();
const userController = require("../controllers/userController");

router.get("/", userController.list);
router.post("/", userController.create);
router.get("/:id", userController.get);

module.exports = router;
```

게시물 라우터와 같은 모양인데 세 줄뿐이다. 유저 컨트롤러에 `update` · `remove` 가 없으니 그에 대응하는 `put` · `delete` 라우트도 없다. 라우터·컨트롤러·서비스가 같은 함수 집합을 공유하는 게 보인다 — 그래서 한 계층에 함수를 더하려면 세 계층을 같이 손봐야 한다.

한 요청을 끝까지 — POST /posts 따라가기

지금까지 본 네 계층이 실제로 어떻게 맞물리는지, 게시물 생성 요청 하나로 정리한다.

```
curl -X POST http://localhost:3000/posts \
  -H "Content-Type: application/json" -d '{"title":"새 글","content":"내용입니다","userId":1}'
```

1. 진입점 — `app.use(express.json())` 이 JSON 본문을 파싱해 `req.body` 에 `{title, content, userId}` 를 채운다. 그리고 `/posts` 로 시작하니 `app.use("/posts", postRoutes)` 가 요청을 게시물 라우터로 넘긴다.
2. 라우터 — 메서드가 `POST`, 경로가 `/` (마운트 뒤이므로 `/posts`)이니 `router.post("/", postController.create)` 가 걸려 `create` 핸들러를 부른다.
3. 컨트롤러 — `create` 가 `req.body` 에서 세 값을 꺼낸다. `title · content` 가 있으니 검증을 통과하고, `postService.create({ title, content, userId })` 를 부른다.
4. 서비스 — `create` 가 `Post.create({ title, content, userId })` 로 모델에게 새 행을 만들라고 시킨다.
5. 모델 — `Post` 가 `posts` 테이블에 행을 추가한다. `userId` 는 관계로 정의된 FK 컬럼에 들어가, 이 글이 1번 유저의 것임을 기록한다.
6. 거슬러 올라가기 — 만들어진 글이 서비스 → 컨트롤러로 돌아오고, 컨트롤러가 `res.status(201).json(post)` 로 응답한다.

목록 조회(`GET /posts`)도 같은 길을 지난다. 차이는 4번에서 `Post.findAll` 에 `include` 가 붙어 작성자가 JOIN되고, 응답의 각 글에 `author: { id, name }` 가 따라온다는 점이다. README의 API 예시에서 게시물에 작성자가 객체로 나오는 건 바로 이 `include` 때문이다.

계층별 역할 한눈에

계층	아는 것	모르는 것	이 장의 대표 코드
라우터	URL + 메서드 → 컨트롤러 함수	그 함수가 하는 일	<code>router.get("/", postController.list)</code>
컨트롤러	<code>req / res</code> , 상태코드, 검증	DB 종류, 질의 방법	<code>res.status(404).json(.)</code>
서비스	비즈니스 로직, 조회 규칙	<code>req / res</code> 가 원지	<code>Post.findAll({.. withAuthor })</code>
모델	테이블 구조, 관계	HTTP, 로직	<code>Post.belongsTo(User, { as: "author" })</code>

흔한 실수

- `include` 의 `as` 와 관계의 `as` 가 다르다. → 모델 입구에서 `as: "author"` 로 관계를 맺었으면, 서비스의 `include` 도 `as: "author"` 여야 한다. 어긋나면 "별칭을 찾을 수 없다"는 에러가 난다.
- 라우터 안 경로를 `/posts` 로 적는다. → 이미 `/posts` 에 마운트했으므로 `/posts/posts` 가 돼 버린다. 라우터 안에 서는 `/`, `/:id` 처럼 마운트 뒤 경로만 적는다.
- 컨트롤러에서 모델을 직접 부른다. → 동작은 하지만 계층이 무너진다. 컨트롤러는 서비스만, 서비스는 모델만 부른다는 한 방향 규칙을 지킨다.
- 서비스에서 `res` 를 만든다. → 서비스는 HTTP를 모른다. 결과(또는 `null`)만 돌려주고, 상태코드는 컨트롤러가 정한다.
- `module.exports` 에 함수를 빠뜨린다. → 라우터가 `postController.update` 를 찾지 못해 `undefined` 가 핸들러로 등록되고, 서버가 뜰 때 에러가 난다. 함수를 더하면 `exports` 목록에도 더한다.

직접 해보기

1. 유저에 수정·삭제를 더해 보자. 유저 서비스에 `update · remove` 를, 유저 컨트롤러에 대응 함수를, 유저 라우터에 `put · delete` 를 추가한다. 게시물 쪽 코드가 그대로 본보기다 — 세 계층을 같이 손봐야 함을 직접 확인하자.
2. GET `/users/:id` 로 한 유저를 조회할 때 그 유저가 쓴 글 목록도 함께 나오게 해 보자. 힌트: 유저 서비스의 `findByPk` 에 `include: [{ model: Post, as: "posts" }]` 를 넣는다. 관계의 `as: "posts"` 는 이미 모델 입구에 있다.
3. 게시물 목록을 작성자로 거를 수 있게 해 보자. GET `/posts?userId=1` 이면 그 유저의 글만 나오도록, 컨트롤러에서 `req.query.userId` 를 꺼내 서비스로 넘기고, 서비스의 `findAll` 에 `where` 를 더한다.

정리

- MVC는 한 파일에 몰려 있던 코드를 라우터 → 컨트롤러 → 서비스 → 모델 네 계층으로 나눈 것이다. 각 계층은 자기 일만 하고, 함수 호출로만 옆 계층과 이어진다.
- 요청은 한 방향으로 흐른다. 라우터가 URL을 컨트롤러에 잇고, 컨트롤러가 HTTP를 다루고, 서비스가 로직을 처리하고, 모델이 DB를 말한다.
- 계층을 잇는 건 `module.exports` 와 `require` 한 쌍이다. 폴더만 나뉘었을 뿐, 함수가 함수를 부르는 평범한 자바스크립트다.
- 작성자를 유저 테이블로 분리하고 `hasMany · belongsTo` 로 관계를 맺으면, 조회 시 `include` 로 `post.author` 를 JOIN해 붙일 수 있다.

다음 장(09 · 인증)에서는 이 게시판에 로그인을 더한다. JWT로 "누가 요청하는지"를 확인해, 로그인한 유저만 글을 쓰게 만든다.